Real-Time UPI Transaction Fraud Detection System

An end-to-end machine learning system for detecting fraudulent UPI payments using graph-based features, ensemble models, and a real-time scoring API.

Domain	Type	Difficulty	Timeline	Stack
Fintech / Payments	ML + Graph + API	Medium	3–4 Days	Python · XGBoost · FastAPI

1 Problem Statement

India's Unified Payments Interface (UPI) processed over **14 billion transactions worth ■20+ lakh crore** in a single month in 2024. The speed that makes UPI powerful also makes it a prime target for fraud — mule account networks, SIM-swap attacks, and account takeovers execute in seconds, far outpacing traditional rule-based detection systems.

Existing fraud detection at many fintechs relies on static rule sets (e.g., flag transactions above ■50,000) that generate excessive false positives and miss sophisticated coordinated fraud. There is a clear need for an adaptive, data-driven system that learns transaction behaviour at the account and network level.

14B+	■20L Cr+	70%+	< 2 sec
Monthly UPI transactions (NPCI, 2024)	Monthly UPI transaction value processed	Fraud cases via mule account networks	Window to detect & block fraud in real-time

2 Project Objectives

01 — Detect fraud in real time
Score each incoming UPI transaction within 100ms using a trained ML model served via a REST API.

02 — Leverage network structure
Model the UPI transaction graph with NetworkX to extract account-level graph features (PageRank, clustering coefficient, degree) that capture mule account behaviour invisible to tabular models alone.

03 — Explain every decision
Use SHAP values to provide top-3 risk factors per flagged transaction — meeting regulatory explainability requirements.

04 **Build production-grade components**
Deliver a FastAPI scoring endpoint, a simulated live transaction stream, and a Streamlit ops dashboard — not just a notebook.

05 **Lay the foundation for scale**
Design the pipeline so the Python-based stream simulator can be swapped for Apache Kafka with minimal code changes.

3 Data Sources & Collection

The project uses a combination of public benchmark datasets (for ground-truth fraud labels) and a synthetic UPI dataset generated programmatically to match the characteristics of Indian payment networks. This approach is standard in fintech risk teams where real transaction data is not publicly available due to privacy regulations.

Source	Type	Purpose	Access
PaySim (Kaggle / GitHub)	Synthetic mobile money transaction logs	Fraud labels & realistic transaction patterns	Free / Public
IEEE-CIS Fraud Detection (Kaggle)	Real anonymised e-commerce transaction data	Additional fraud signal for model training	Free / Public
Synthetic UPI Dataset (self-generated)	Python Faker + custom fraud pattern scripts	UPI-specific graph structure, VPA IDs, velocity features	Self-generated
NPCI UPI Statistics (npci.org.in)	Aggregate volume & value reports (public)	Calibrate synthetic data to real UPI scale	Free / Public
RBI Payment System Reports	Official fraud statistics by payment type	Fraud rate benchmarks for evaluation targets	Free / Public

Synthetic data generation approach: Using Python's Faker library, 5,000 virtual UPI accounts are created with realistic VPA IDs (e.g., name@upi). 50,000 transactions are generated over a 90-day window with realistic time-of-day distributions. Fraud is injected by designating 10% of accounts as mule accounts that exhibit characteristic patterns: receiving many small credits from multiple senders then executing a single large outward transfer.

4 Methodology

Phase 1
Data Generation & Preprocessing
Generate synthetic UPI transaction dataset. Clean, merge PaySim labels. Compute velocity features using rolling time windows (1h, 6h, 24h) per account. Handle class imbalance with SMOTE and scale_pos_weight.

Phase 2
Graph Feature Engineering
Build a directed transaction graph with NetworkX where nodes are UPI accounts and edges are transactions weighted by amount. Extract per-node features: in-degree, out-degree, PageRank score, local clustering coefficient, and betweenness centrality. These capture mule account topology.

Phase 3

Model Training & Evaluation

Train XGBoost classifier on combined tabular + graph feature matrix. Train Isolation Forest as unsupervised baseline. Evaluate on PR-AUC, F1 at operating threshold (Precision >= 0.85). Select final threshold balancing false positive rate acceptable to ops team.

Phase 4

Explainability & Validation

Apply SHAP TreeExplainer to generate per-prediction explanations. Produce SHAP waterfall plots for individual transactions and a global feature importance summary. Validate that top features align with domain knowledge (velocity, PageRank, amount deviation).

Phase 5

API & Dashboard Deployment

Serve model via FastAPI: POST /predict (returns score + SHAP factors), GET /transactions (recent flagged txns from SQLite). Build Streamlit dashboard: live fraud queue, hourly fraud rate chart, network graph visualiser (PyVis), manual transaction scoring form.

Phase 6

Simulated Stream & Documentation

simulate_stream.py fires one synthetic transaction per second at the FastAPI endpoint — creating a live updating dashboard without Kafka complexity. Architecture is designed so this script is swappable with a Kafka consumer. Write README, architecture diagram, and model card.

5 System Architecture

The system follows a layered architecture separating data ingestion, feature computation, model inference, and presentation.

Data Layer	Synthetic UPI dataset (Faker) · PaySim benchmark · IEEE-CIS dataset · SQLite transaction store
Feature Engineering Layer	Tabular features (velocity, amount stats, time) · Graph features (NetworkX — PageRank, degree, clustering)
Model Layer	XGBoost classifier · Isolation Forest baseline · SHAP explainability · Calibrated probability scores
Serving Layer	FastAPI REST API · /predict endpoint · /transactions endpoint · SQLite result store
Presentation Layer	Streamlit dashboard · Live fraud queue · Network graph (PyVis) · Manual scoring form
Simulation Layer	simulate_stream.py · 1 txn/sec synthetic stream · Designed for Kafka swap-in

6 Technology Stack

Category	Libraries / Tools	Purpose
Data Generation	Faker, NumPy, Pandas	Synthetic UPI transaction dataset
Graph Analysis	NetworkX	Transaction graph + node feature extraction
ML Modelling	XGBoost, Scikit-learn	Fraud classifier + anomaly baseline
Explainability	SHAP	Per-prediction feature attribution

Experiment Tracking	MLflow	Model versioning, metrics, artefacts
API Serving	FastAPI, Uvicorn	Real-time scoring REST endpoint
Database	SQLite / PostgreSQL	Transaction and prediction storage
Dashboard	Streamlit, PyVis	Ops dashboard + network visualisation
Containerisation	Docker, docker-compose	Reproducible local deployment
Stream Simulation	Python threading	1 txn/sec live data simulation
Future Streaming	Apache Kafka, confluent-kafka	Production-scale ingestion (planned)

7 Evaluation Metrics & Success Criteria

PR-AUC	**> 0.85**	Primary metric for imbalanced fraud data. Measures model ability to rank fraudulent transactions above legitimate ones.
Precision @ threshold	**> 0.85**	At the operating threshold, at least 85% of flagged transactions should be true fraud — keeping the false positive burden on ops teams low.
Recall	**> 0.70**	Capture at least 70% of all fraud cases in the dataset — acceptable trade-off against precision for a first-generation model.
API latency (p95)	**< 100ms**	The /predict endpoint must score a transaction within 100ms at p95 — the realistic SLA for a payment gateway integration.
SHAP coverage	**Top 3 factors per flag**	Every flagged transaction must have SHAP-derived top-3 risk factors returned in the API response for explainability compliance.

8 Deliverables

GitHub Repository. Full source code, structured as a Python package with clear module separation (data/, features/, models/, api/, dashboard/).

Trained Model Artefact. XGBoost model saved via joblib with MLflow tracking. Includes model card documenting training data, features, performance metrics, and known limitations.

FastAPI Application. Dockerised REST API with /predict and /transactions endpoints. Includes Swagger UI documentation auto-generated by FastAPI.

Streamlit Dashboard. Live ops dashboard deployed on Streamlit Cloud (free tier). Shows real-time fraud queue, hourly fraud rate trend, account network graph, and manual scoring form.

Project README. Architecture diagram, setup instructions, demo GIF, results table, and a section explaining how to replace simulate_stream.py with Kafka.

Evaluation Report. Jupyter notebook with PR-AUC curves, confusion matrix, SHAP summary plot, top 10 most predictive features, and ablation study (tabular-only vs tabular+graph).

9 4-Day Implementation Timeline

Day	Focus Area	Key Outputs
Day 1	Data generation & feature engineering	Synthetic UPI dataset (50K txns) · tabular features · NetworkX graph · merged feature matrix
Day 2	Model training & explainability	XGBoost + Isolation Forest trained · SHAP plots · PR-AUC evaluation · MLflow experiment logged
Day 3	FastAPI + Streamlit dashboard	Scoring API live · dashboard with charts + graph viz · manual scoring form working
Day 4	Stream simulation, polish & README	simulate_stream.py running · dashboard auto-refreshing · Docker compose · README + demo GIF

10 Future Scope (Extended Version)

- Replace simulate_stream.py with Apache Kafka + Faust for true real-time ingestion at production scale.
- Upgrade from NetworkX graph features to a full GraphSAGE GNN (PyTorch Geometric) trained end-to-end on the transaction graph.
- Add Redis velocity cache for sub-millisecond lookups of rolling account statistics.
- Integrate device fingerprinting embeddings as additional features (device ID, IP, OS, geolocation delta).
- Deploy on Kubernetes with Helm chart, Prometheus metrics, and Grafana alerting dashboard.
- Add Evidently AI for automated data drift monitoring and retraining triggers.
- Implement champion-challenger A/B testing between model versions in production.

This proposal outlines a focused, deliverable, and technically credible project that mirrors real fraud detection systems in production at Indian fintech companies. The 4-day scoped version is fully portfolio-ready; the future scope section demonstrates architectural awareness of production requirements.